



CEBU INSTITUTE OF TECHNOLOGY – UNIVERSITY
COLLEGE OF ENGINEERING AND ARCHITECTURE
DEPARTMENT OF COMPUTER ENGINEERING

CPE463:
COMPUTER ARCHITECTURE AND ORGANIZATION

PROJECT PROPOSAL
ATM Simulator

Prepared By:

Cañete, Lemar
Dela Cerna, Neil Franz
Divera, Francesca Drinne
Luzares, Benz Gerald
Macapaz, Chaz
Ocon, Christian Edison
Rubi, Alan Anthony



1. Project Overview

Objective:

This project simulates a basic ATM login system using assembly language. It prompts the user for a numeric ID and password and verifies them against predefined credentials stored in memory. This simulates the user authentication process in real-world ATM systems and demonstrates core low-level programming concepts such as memory access, loops, and input/output handling.

Features:

- Displays a welcome message to the user.
- Accepts a user ID and password as numeric input.
- Verifies the ID against a list of stored IDs.
- Matches the entered password with the corresponding one stored in memory.
- Displays "Access Allowed" or "Access Denied" messages based on credentials.
- Loops back to the login screen upon incorrect credentials.

2. Technical Design and Implementation

Code Features Implementation

Feature	Implementation
Data Transfer Instructions (e.g., MOV, PUSH, POP)	Values are moved between memory locations and CPU registers to facilitate ID and password handling, data setup, and flow control using standard data transfer instructions.
Arithmetic Operations (e.g., ADD, SUB, MUL, DIV)	Arithmetic division is used to compute positions within arrays, particularly when validating user credentials based on their location in memory.
Logical Operations (e.g., AND, OR, XOR)	Logical XOR is applied to implement a basic encryption and decryption mechanism for securing user-entered PINs in memory.
Loops and Conditional Branching (e.g., CMP, Jxx)	Looping and decision-making are performed using comparisons and conditional jumps, enabling the program to repeatedly prompt users and validate login credentials.
Memory Access Using Addressing Modes (e.g., direct, indirect, indexed)	The program accesses data from memory using indexed addressing to iterate through stored user IDs and corresponding credentials.
Input/Output Operations (e.g., interrupts or user interaction)	User interaction is simulated with high-level macros that scan for numeric input and display messages, emulating real-world input/output operations in the context of an ATM interface.